

◆ 1. lambda (Start here)

✓ What is lambda?

A **lambda function** is a **small anonymous function** (no name).

```
lambda x: x * 2
```

👉 Same as:

```
def double(x):  
    return x * 2
```

◆ Example

```
f = lambda x: x + 5  
print(f(3)) # 8
```

◆ When to use?

- Short, one-line functions
- Used inside `map()`, `filter()`, `apply()`

◆ 2. map() (Apply function to each element)

✓ What is map()?

Applies a function to **every element of an iterable**

◆ Syntax

```
map(function, iterable)
```

◆ Example

```
nums = [1, 2, 3, 4]
```

```
result = map(lambda x: x * 2, nums)
```

```
print(list(result))
```

◆ Output

```
[2, 4, 6, 8]
```

◆ Internal idea

```
result = []  
for x in nums:  
    result.append(x * 2)
```

◆ 3. map() with multiple lists

```
a = [1, 2, 3]  
b = [4, 5, 6]
```

```
result = map(lambda x, y: x + y, a, b)  
print(list(result))
```

👉 Output:

```
[5, 7, 9]
```

◆ 4. apply() (Pandas only)

👉 This is where things change ⚠️

apply() is used with **Pandas DataFrames/Series**

◆ Example

```
import pandas as pd
```

```
df = pd.DataFrame({'A': [1, 2, 3]})
```

```
df['A'] = df['A'].apply(lambda x: x * 2)
```

```
print(df)
```

◆ Output

A
0 2
1 4
2 6

◆ 5. map() vs apply() (IMPORTANT)

Feature	map()	apply()
Works on Series (column)	Series & DataFrame	
Speed	Faster	Slower
Flexibility	Less	More powerful
Use case	Simple mapping	Complex operations

◆ 6. DataFrame apply() (Advanced)

```
df = pd.DataFrame({  
    'A': [1, 2],  
    'B': [3, 4]  
})
```

Row-wise operation

```
df['sum'] = df.apply(lambda row: row['A'] + row['B'], axis=1)
```

👉 Output:

```
  A B sum  
0 1 3  4  
1 2 4  6
```

◆ 7. When to Use What?

✓ Use lambda:

- When function is small and simple

✓ Use map():

- Apply function to each value in a list or column

✓ Use apply():

- When working with **Pandas**
- When logic is more complex (row/column based)

◆ 8. Best Practice (IMPORTANT)

In Pandas, prefer **vectorized operations** 🙌

`df['A'] = df['A'] * 2` # ✓ faster than apply/map

◆ 🔥 Final Intuition

- `lambda` → *tiny function*
- `map()` → *apply function to each item*
- `apply()` → *powerful version for Pandas*

◆ One-Line Summary

lambda creates small functions, map() applies them to iterables, and apply() applies them to Pandas data structures.

◆ 1. What is filter()?

👉 filter() is used to **select elements** from a collection based on a condition.

- Keeps values that return **True**
- Removes values that return **False**

◆ 2. Syntax

filter(function, iterable)

- function → condition (returns True/False)
- iterable → list, tuple, etc.

◆ 3. Basic Example

Keep even numbers:

```
nums = [1, 2, 3, 4, 5, 6]
```

```
result = filter(lambda x: x % 2 == 0, nums)
```

```
print(list(result))
```

◆ Output

```
[2, 4, 6]
```

◆ 4. How it works internally

```
result = []
```

```
for x in nums:
```

```
    if x % 2 == 0:
```

```
        result.append(x)
```

◆ 5. Another Example

Keep numbers greater than 10

```
nums = [5, 12, 8, 20, 3]
```

```
result = filter(lambda x: x > 10, nums)
```

```
print(list(result))
```

👉 Output:

```
[12, 20]
```

◆ 6. Using normal function instead of lambda

```
def is_even(x):  
    return x % 2 == 0
```

```
nums = [1, 2, 3, 4]
```

```
result = filter(is_even, nums)
```

```
print(list(result))
```

◆ 7. Important Note

👉 filter() returns a **filter object (iterator)**

So you usually convert it:

```
list(result)
```

◆ 8. filter() vs map()

Feature	map()	filter()
---------	-------	----------

Purpose	Transform values	Select values
---------	------------------	---------------

Output	Modified values	Subset of values
--------	-----------------	------------------

◆ 🔥 Key Intuition

- `map()` → *change values*
- `filter()` → *pick values*

◆ One-Line Summary

`filter()` keeps only the elements that satisfy a condition.